

CC3104 — Aprendizaje por Refuerzo

Examen Práctico: Auditoría del agente de gestión de inventario

Grupo 4 — La aproximación de función

Nombre	Carnet	Usuario Git
Edwin Jose Gabriel De Leon Garcia	22809	EJGDLG
Gustavo Adolfo Cruz Bardales	22779	G2309
Josué Emanuel Say Garcia	22801	JosueSay
Mathew Alexander Cordero Aquino	22982	donmatthiuz
Carlos Alberto Valladares Guerra	221164	vgcarlo1

8 de septiembre de 2026

Índice

1. Análisis del componente asignado	1
Metodología de medición	1
1.1. Entregable 4.1: el problema de escala	2
1.2. Entregable 4.2: interacciones que el modelo lineal no captura	4
1.3. Entregable 4.3: versión mejorada de <code>preprocess_state</code>	4
1.4. Entregable 4.4: comparación de error cuadrático medio	7
2. Respuestas a las preguntas de integración	10
2.1. Pregunta 1: Diagnóstico sistémico	10
2.2. Pregunta 2: Causa raíz	11
2.3. Pregunta 3: Plan de corrección mínimo viable	13
2.4. Pregunta 4: Compatibilidad de correcciones	14
3. Reflexión grupal	19
A. Anexo: uso de IA generativa	20

1. Análisis del componente asignado

Metodología de medición

Para poder medir el error del aproximador se necesita un objetivo de referencia. Como la función de transición del sistema es determinista, el MDP se resolvió de forma exacta por iteración de valor con $\gamma = 0,99$, obteniendo $Q^*(s, a)$ para todos los estados alcanzables. Se usó la función de recompensa *original* del sistema entregado, no una corregida.

La especificación declara 200 estados (10 niveles de inventario $\times$ 5 de vencimiento $\times$ 4 de demanda), pero `transition` devuelve `inventory + action - daily_demand`, que no es múltiplo de 10 salvo cuando la demanda lo es. La cerradura del espacio de estados bajo la función de transición contiene **4 367 estados**, no 200. El agente pasa la mayor parte del tiempo en estados que la especificación no declara.

Todos los ajustes se hacen sobre los 4 367 estados alcanzables, reservando 24 estados declarados como conjunto de prueba (Sección 1.4).

1.1. Entregable 4.1: el problema de escala

Qué componente domina el gradiente

El aproximador es lineal, $Q(s, a; w) = w_a^\top \phi(s)$, y su actualización es la regla LMS / semi-gradiente:

$$w_a \leftarrow w_a + \alpha [y - w_a^\top \phi(s)] \nabla_{w_a} Q(s, a; w), \quad \nabla_{w_a} Q(s, a; w) = \phi(s).$$

El gradiente *es* el vector de características. Por lo tanto la magnitud del paso que recibe la coordenada i es proporcional a $\phi_i(s)$, y el aporte de esa coordenada a la norma del gradiente es $\phi_i(s)^2 / \|\phi(s)\|^2$. Con

$$\begin{aligned} \phi(s) &= [\text{inventory}, \text{days_to_expiry}, \text{demand_idx}], \\ \text{inventory} &\in [0, 100], \quad \text{days} \in \{1, 7, 14, 30, 60\}, \quad \text{demand_idx} \in \{0, 1, 2, 3\}, \end{aligned}$$

el promedio sobre los 200 estados declarados da:

Componente	Aporte a $\ \phi\ ^2$ (%)
inventory	74.95
days_to_expiry	24.96
demand_idx	0.09

Cuadro 1: Participación de cada característica en la norma del gradiente.

El nivel de inventario domina el gradiente con el 75 % y la demanda aporta el 0,09 %, unas 815 veces menos. La demanda determina si un producto se agota, y es la componente que menos peso tiene en el gradiente. El peso asociado a la demanda se mueve tres órdenes de magnitud más lento que el del inventario: para cuando el aproximador ha aprendido algo sobre la demanda, el resto de los pesos ya convergió a una solución que la ignora. El sesgo no es de capacidad sino de ritmo de aprendizaje efectivo.

Divergencia del entrenamiento

Para LMS con paso fijo α , la condición de convergencia en media es

$$0 < \alpha < \frac{2}{\|\phi(s)\|^2}.$$

El máximo observado es $\|\phi\|_{\text{máx}}^2 = 11\,709$ (estado (90, 60, crítico)), de donde

$$\alpha_{\text{máx}} = \frac{2}{11\,709} = 1,708 \times 10^{-4}.$$

α	Características originales	Características propuestas
10^{-2}	diverge (overflow)	converge ($\ w\ _\infty \approx 3626$)
10^{-3}	diverge (overflow)	—
$1,4 \times 10^{-4}$	converge	—
10^{-4}	converge	—
5×10^{-2}	—	converge (≈ 3710)
10^{-1}	—	converge (≈ 3720)

Cuadro 2: Estabilidad empírica del ajuste por SGD. Con las características originales, el α del código diverge; con las propuestas, α puede ser hasta 100 veces mayor sin problemas.

El código entregado usa $\alpha = 0,01$ por defecto en **update**, es decir **58,5 veces por encima de la cota de estabilidad**. Se verificó empíricamente ajustando w por SGD sobre los estados de entrenamiento:

Adicionalmente, el número de condición de la matriz de Gram $\frac{1}{N}\Phi^\top\Phi$ pasa de $\kappa = 1888$ con las características crudas a $\kappa = 26,3$ con las mismas tres características normalizadas. Como la velocidad de convergencia de un método de primer orden escala con κ , la normalización sola acelera la convergencia en aproximadamente dos órdenes de magnitud.

Defecto adicional: ausencia de intercepto

ϕ no incluye un término constante y **weights** se inicializa en cero. Por lo tanto, para el estado $(0, d, \text{bajo})$ se tiene $\phi = (0, d, 0)$, y para $(0, \cdot, \text{bajo})$ con d pequeño el vector es casi nulo: el modelo está forzado a pasar por el origen. Como se muestra en la Sección 1.4, esto por sí solo hace que el R^2 del aproximador original sea $-132,7$, o sea peor que predecir constantemente el promedio de Q^* .

Normalización propuesta por componente

Cada componente se lleva a $[0, 1]$ usando una constante que proviene del propio dominio, no de un escalado genérico:

Componente	Transformación	Justificación
inventory	$\text{inv}/100 \in [0, 1]$	100 es la capacidad real impuesta por el $\min(100, \cdot)$ de transition ; no es una constante arbitraria.
days_to_expiry	$(d - 1)/59 \in [0, 1]$ más el indicador $\mathbf{1}[d \leq 7]$	El rango declarado es $[1, 60]$. El indicador replica la forma exacta de la penalización $-10 \cdot \mathbf{1}[d \leq 7]$: sin él, ninguna combinación lineal de d puede representar un escalón.
demand_level	<i>One-hot</i> con bajo como base	El código ordinal 0, 1, 2, 3 impone dos supuestos falsos: que la demanda tiene orden lineal en el valor y que el espaciado es uniforme. La demanda real es 5, 15, 25, 40, cuyos incrementos son 10, 10, 15. Además bajo =0 anula la característica y hace indistinguibles dos estados con demanda baja.

Cuadro 3: Normalización propuesta. Con ella $\|\phi\|^2 \leq 5,93$ y $\alpha_{\max} = 0,337$, frente a $1,708 \times 10^{-4}$ del original.

1.2. Entregable 4.2: interacciones que el modelo lineal no captura

Un aproximador lineal sobre $\phi = [\text{inv}, d, \text{dem}]$ es aditivo: el efecto marginal de cada variable es constante y no depende de las otras, $\partial Q / \partial \text{inv} = w_{a,1}$ sin importar el valor de la demanda. En gestión de inventario farmacéutico eso es falso. Las dos interacciones más importantes son:

Interacción 1: inventario $\times$ demanda (riesgo de desabasto). El riesgo de *stockout* no depende del inventario ni de la demanda por separado, sino de su **cociente**: los días de cobertura

$$\text{cobertura}(s) = \frac{\text{inventory}}{\text{daily_demand}}.$$

Un inventario de 30 unidades es abundante con demanda baja (6 días de cobertura) y crítico con demanda crítica (0,75 días). Un modelo aditivo asigna a $\text{inv} = 30$ el mismo aporte al valor en ambos casos. Un cociente no es expresable como $w_1 x_1 + w_2 x_2$ para ningún (w_1, w_2) : es una imposibilidad representacional, no un ajuste imperfecto. Se agrega explícitamente como característica, saturada a 20 días y normalizada, junto con la brecha inmediata $\text{máx}(0, \text{dem} - \text{inv})/40$, que marca los estados en los que el desabasto ocurre en el paso siguiente.

Interacción 2: inventario $\times$ vencimiento $\times$ demanda (merma). Las unidades que van a vencerse son las que no alcanzarán a venderse antes del vencimiento:

$$\text{merma}(s) = \text{máx}(0, \text{inventory} - \text{daily_demand} \cdot \text{days_to_expiry}).$$

Es un producto seguido de una rectificación. Los 41 productos vencidos por semana en producción son exactamente esta cantidad, y el aproximador original no tiene ninguna característica que se le parezca. Con $d = 60$ y demanda crítica no hay merma con ningún inventario; con $d = 1$ y demanda baja casi todo el inventario se pierde. Un término aditivo en inv y otro en d no pueden distinguir esos dos casos. Se agrega también $\text{inv}_n \cdot \mathbf{1}[d \leq 7]$, que captura que el costo del inventario alto se invierte de signo cuando el producto está por vencer.

1.3. Entregable 4.3: versión mejorada de preprocess_state

El vector de características propuesto tiene 12 componentes:

Listing 1: código/preprocesamiento_mejorado.py

```
1 DEMAND_MAP = {"bajo": 5, "medio": 15, "alto": 25, "critico": 40}
2 DEMAND_ORDER = ["bajo", "medio", "alto", "critico"]
3 INVENTORY_MAX = 100.0
4 DEMAND_MAX = 40.0
5 EXPIRY_MAX = 60.0
6 EXPIRY_THRESHOLD = 7
7 SUPPLY_CAP = 20.0
8
9 FEATURE_NAMES = [
10     "bias",
11     "inv_n",
12     "inv_n2",
13     "exp_n",
14     "near_expiry",
15     "dem_medio",
16     "dem_alto",
17     "dem_critico",
18     "dias_cobertura",
```

#	Nombre	Definición y rol
0	bias	1 — intercepto; permite representar estados con ϕ casi nulo.
1	inv_n	$\text{inv}/100$
2	inv_n2	$(\text{inv}/100)^2$ — no linealidad del costo de almacenamiento y saturación en la capacidad.
3	exp_n	$(d - 1)/59$
4	near_expiry	$\mathbf{1}[d \leq 7]$ — discontinuidad exacta de reward.
5-7	dem_*	<i>one-hot</i> de demanda (medio, alto, crítico; base bajo).
8	dias_cobertura	$\min(\text{inv}/\text{dem}, 20)/20$ — interacción 1.
9	riesgo_merma	$\max(0, \text{inv} - \text{dem} \cdot d)/100$ — interacción 2.
10	inv_x_near_expiry	$\text{inv}_n \cdot \mathbf{1}[d \leq 7]$
11	brecha_stockout	$\max(0, \text{dem} - \text{inv})/40$

Cuadro 4: Características propuestas. Todas están en $[0, 1]$, de modo que $\|\phi\|^2 \leq 12$ por construcción (máximo observado: 5,93).

```

19     "riesgo_merma",
20     "inv_x_near_expiry",
21     "brecha_stockout",
22 ]
23 N_FEATURES = len(FEATURE_NAMES)
24
25
26 def _normalizar_demanda(demand_level):
27     if isinstance(demand_level, str):
28         return demand_level.replace("í", "i")
29     return demand_level
30
31
32 def preprocess_state(state):
33     inventory, days_to_expiry, demand_level = state
34     demand_level = _normalizar_demanda(demand_level)
35     daily_demand = DEMAND_MAP[demand_level]
36
37     inv = float(min(max(inventory, 0.0), INVENTORY_MAX))
38     days = float(min(max(days_to_expiry, 1.0), EXPIRY_MAX))
39
40     inv_n = inv / INVENTORY_MAX
41     exp_n = (days - 1.0) / (EXPIRY_MAX - 1.0)
42     near_expiry = 1.0 if days <= EXPIRY_THRESHOLD else 0.0
43     dias_cobertura = min(inv / daily_demand, SUPPLY_CAP) / SUPPLY_CAP
44     riesgo_merma = max(0.0, inv - daily_demand * days) / INVENTORY_MAX
45
46     features = [0.0] * N_FEATURES
47     features[0] = 1.0
48     features[1] = inv_n
49     features[2] = inv_n * inv_n
50     features[3] = exp_n
51     features[4] = near_expiry
52     features[5] = 1.0 if demand_level == "medio" else 0.0
53     features[6] = 1.0 if demand_level == "alto" else 0.0
54     features[7] = 1.0 if demand_level == "critico" else 0.0
55     features[8] = dias_cobertura
56     features[9] = riesgo_merma

```

```

57     features[10] = inv_n * near_expiry
58     features[11] = max(0.0, daily_demand - inv) / DEMAND_MAX
59     return features
60
61
62 def preprocess_state_original(state):
63     inventory, days_to_expiry, demand_level = state
64     demand_level = _normalizar_demanda(demand_level)
65     demand_idx = {"bajo": 0, "medio": 1, "alto": 2, "critico": 3}[demand_level]
66     return [float(inventory), float(days_to_expiry), float(demand_idx)]
67
68
69 class LinearApproximator:
70     def __init__(self, n_features=N_FEATURES, n_actions=6, q_init=0.0,
71                 normalize_step=True):
72         self.n_features = n_features
73         self.n_actions = n_actions
74         self.normalize_step = normalize_step
75         self.weights = [[0.0] * n_features for _ in range(n_actions)]
76         if q_init != 0.0:
77             for a in range(n_actions):
78                 self.weights[a][0] = q_init
79
80     def predict(self, features, action):
81         w = self.weights[action]
82         return sum(w[i] * features[i] for i in range(self.n_features))
83
84     def update(self, features, action, target, alpha=0.05):
85         error = target - self.predict(features, action)
86         if self.normalize_step:
87             norm2 = sum(f * f for f in features)
88             step = alpha / norm2 if norm2 > 0.0 else 0.0
89         else:
90             step = alpha
91         w = self.weights[action]
92         for i in range(self.n_features):
93             w[i] += step * error * features[i]
94         return error
95
96
97 def cota_estabilidad(features):
98     norm2 = sum(f * f for f in features)
99     return float("inf") if norm2 == 0.0 else 2.0 / norm2

```

Cambios al aproximador

1. **Paso normalizado.** `update` usa $\alpha_{\text{eff}} = \alpha / \|\phi(s)\|^2$ (regla LMS normalizada), que garantiza estabilidad para cualquier escala de entrada.
2. **Inicialización optimista opcional.** Como ahora existe el intercepto, fijar $w_{a,0} = q_{\text{init}}$ produce $Q(s, a) \approx q_{\text{init}}$ para todo s , lo que da exploración dirigida sin tocar ε . Con el ϕ original esto era imposible.
3. **Compatibilidad.** La firma de `preprocess_state` y de `predict/update` no cambia; solo cambia `n_features` de 3 a 12. El resto del sistema no requiere modificaciones.

¿Es suficiente el aproximador lineal?

Sí, para este dominio: $R^2 = 0,967$ sobre estados no vistos (entregable 4.4). Las no linealidades de este MDP son conocidas y de forma cerrada — un escalón en $d = 7$, un producto rectificado para la merma, un cociente para la cobertura y una saturación en la capacidad. Cuando la no linealidad se conoce, incluirla como característica y ajustar un modelo lineal encima conserva convergencia garantizada bajo aproximación lineal, pesos interpretables y costo de cómputo trivial.

Se requeriría aproximación no lineal si el estado se extendiera con variables cuya interacción no se conoce de antemano: estacionalidad, sustitución entre productos o demanda estocástica con correlación temporal.

El número de condición del bloque completo de 12 características es $\kappa \approx 2,1 \times 10^3$, similar al del original, porque `riesgo_merma` es cero en la mayoría de los estados y queda casi colineal con el resto. La normalización arregla la *escala* (que es lo que causa la divergencia), no elimina la colinealidad. Se mitiga con regularización ridge ($\lambda = 10^{-8}$ en los ajustes reportados) o eliminando la característica 10, que es la de menor aporte marginal.

1.4. Entregable 4.4: comparación de error cuadrático medio

Protocolo

- Objetivo: $Q^*(s, a)$ exacto por iteración de valor ($\gamma = 0,99$, tolerancia 10^{-10}) sobre los 4 367 estados alcanzables.
- Conjunto de prueba: **24 estados representativos**, tomados de forma estratificada sobre los 200 estados declarados para cubrir todo el rango de inventario, los cinco niveles de vencimiento y los cuatro de demanda.
- Conjunto de entrenamiento: los 4 343 estados restantes. Los estados de prueba *no* se usan para ajustar.
- Ajuste: mínimos cuadrados exacto por acción (ecuaciones normales con ridge $\lambda = 10^{-8}$). Se usa la solución óptima en lugar de SGD para que la comparación mida **capacidad de representación** y no calidad del optimizador; el efecto sobre el optimizador se reporta por separado en 4.1.
- Métrica: $\text{MSE} = \frac{1}{|S| \cdot |A|} \sum_s \sum_a (Q^*(s, a) - w_a^\top \phi(s))^2$.

Resultado global

Aproximador	MSE entren.	MSE prueba	R^2 prueba
Original (3 caract., sin intercepto)	1,635,206.0	2,759,751.7	-132.716
Original + intercepto (4 caract.)	1,041.6	990.1	0.952
Propuesto (12 caract.)	318.2	678.4	0.967

Cuadro 5: Comparación de MSE. La varianza de Q^* es $\sigma^2 = 20\,638,8$; $R^2 = 1 - \text{MSE}/\sigma^2$. Reducción del MSE de prueba respecto al original: **99,98 %**.

La fila intermedia (características originales más un intercepto) es una ablación. El salto de $2,76 \times 10^6$ a 990 se debe casi por completo al intercepto faltante, no a las características nuevas. Aislado ese efecto, las interacciones del entregable 4.2 aportan una **reducción adicional del 31,5 %** del MSE de prueba ($990 \rightarrow 678$) y suben el R^2 de 0,952 a 0,967.

El ajuste original es *peor que la predicción constante* ($R^2 < 0$). Un aproximador con R^2 negativo alimentando una política greedy $\arg \max_a Q(s, a)$ no ordena las acciones mejor que el azar en los estados donde ϕ es pequeño, que son los de inventario bajo, donde ocurren los desabastos.

Detalle por estado (24 estados de prueba)

Inv.	d	Demanda	$\max_a Q^*$	a^*	MSE original	MSE propuesto
0	1	bajo	3,767.5	50	13,610,118.3	233.3
0	14	bajo	3,826.1	50	9,319,062.9	216.8
0	60	bajo	4,174.6	50	887,390.8	653.9
10	7	medio	3,762.6	50	6,820,040.4	327.4
10	30	medio	3,961.0	50	2,497,293.7	211.5
20	1	medio	3,772.5	50	7,258,968.9	506.5
20	14	alto	3,813.7	50	2,344,597.2	105.3
20	60	alto	4,162.2	50	335,822.6	457.4
30	7	alto	3,767.6	50	2,600,258.0	982.1
30	30	crítico	3,895.1	50	2,425.2	1,067.7
40	1	crítico	3,726.0	50	1,196,949.6	264.0
40	14	crítico	3,784.5	50	210,012.0	1,988.1
50	1	bajo	3,797.5	50	6,165,322.4	711.7
50	14	bajo	3,856.0	50	3,409,680.5	2.9
50	60	bajo	4,204.5	50	69,797.3	94.4
60	7	medio	3,797.5	50	1,998,061.6	293.5
60	30	medio	3,995.9	50	146,148.3	140.5
70	1	medio	3,800.0	50	2,222,812.5	338.2
70	14	alto	3,856.0	50	121,051.8	31.5
70	60	alto	4,204.5	50	3,107,585.7	281.0
80	7	alto	3,800.0	50	177,655.3	503.7
80	30	crítico	3,993.4	50	1,300,516.0	1,923.6
90	1	crítico	3,800.0	50	444.9	3,752.1
90	14	crítico	3,858.5	50	432,025.0	1,194.6

Cuadro 6: MSE por estado, promediado sobre las 6 acciones.

El aproximador propuesto gana en 23 de los 24 estados.

Lectura de la tabla

- El aproximador propuesto gana en **23 de 24** estados. El único estado donde pierde es (90, 1, crítico), con inventario máximo y vencimiento inmediato: ahí $\|\phi_{\text{orig}}\|$ es grande y el modelo sin intercepto acierta la escala de Q^* por coincidencia de magnitud.
- El error del original crece a medida que el inventario baja (1.4×10^7 en $\text{inv} = 0$), exactamente el efecto del intercepto ausente descrito en 4.1.
- El error residual del modelo propuesto se concentra en los estados de demanda crítica con vencimiento cercano ((90, 1, crítico): 3752). Son los estados de frontera del dominio y los peor cubiertos por el conjunto de entrenamiento.

- La acción óptima es `pedir 50` en los 24 estados de prueba. Con la función de recompensa original, pedir el máximo *sí* es la política óptima del MDP: el “94 % de estados con `pedir_50`” de las métricas de entrenamiento no es un fallo de aprendizaje, sino el resultado de una recompensa mal especificada. Ninguna corrección del preprocesamiento lo modifica.

Reproducibilidad

Listing 2: Reproducción de los resultados

```
python3 codigo/experimento_mse.py
```

No requiere dependencias externas (Python puro). Los valores crudos, incluidos los pesos ajustados, quedan en `resultados/resultados_44.json`.

2. Respuestas a las preguntas de integración

2.1. Pregunta 1: Diagnóstico sistémico

Con base en el MDP estocástico formulado por el Grupo 1, la función de recompensa multicomponente diseñada por el Grupo 2, la ablación empírica y curvas proyectadas por el Grupo 6, y el análisis de aproximación y estabilidad de nuestro Grupo 4, **predecimos categóricamente que el agente NO aprenderá una política mejor si únicamente se aplican las correcciones del MDP y de la función de recompensa sin modificar el algoritmo de aprendizaje ni la estrategia de exploración.**

Aunque la corrección conjunta del MDP y de la señal de recompensa repara la especificación matemática del entorno y reorienta teóricamente los incentivos hacia políticas de inventario sostenibles, el proceso de optimización del agente colapsa inevitablemente debido a tres fallas estructurales y cuantitativas:

1. **Eficacia teórica aislada de la recompensa de G2:** Bajo la función corregida del Grupo 2 ($R_{\text{corregida}}$), en una simulación de 1000 episodios y 30 pasos por episodio (semilla 3104), la política viciada de “pedir siempre 50” es severamente penalizada, acumulando una recompensa media de -842.29 ($\sigma = 476.22$), con un inventario promedio saturado de 97.98 unidades y 18.11 pasos por episodio en ventana de vencimiento. Por el contrario, una política balanceada alcanza una recompensa acumulada media positiva de $+144.40$ ($\sigma = 67.81$), reduciendo el inventario promedio a 5.76 unidades, los pasos en riesgo de vencimiento a 7.48 y manteniendo 0.00 pasos con desabasto (*stockouts*). Esto evidencia que la señal de recompensa corregida contiene el gradiente de incentivos adecuado para guiar al agente.
2. **Explosión del espacio de búsqueda de G1 frente a la inestabilidad de G3 y G4:** Para restaurar la propiedad de Markov y dotar de dinamismo temporal a la demanda, el Grupo 1 amplió la tupla de estado a 5 variables: $s_t = (I_t, E_t, L_t, T_t, D_t)$, incorporando las unidades en tránsito ($T_t \in \{0, 10, 20, 30, 40, 50\}$) y la tendencia de demanda ($D_t \in \{\text{bajando, estable, subiendo}\}$). Dicha corrección expande el espacio de estados de 200 nominales a 3600 estados discretos, y el espacio de pares estado-acción de 1200 a 21600 (un factor de multiplicación exacto de $18\times$).

Si se mantiene el algoritmo de aprendizaje original:

- El valor fijo $\alpha = 0.9$ es catastrófico para la convergencia. Como demostró analíticamente nuestro Grupo 4 mediante la condición LMS ($0 < \alpha < \frac{2}{\|\phi(s)\|^2}$), el máximo paso admisible antes de divergir numéricamente es $\alpha_{\text{máx}} \approx 1.708 \times 10^{-4}$. Un valor $\alpha = 0.9$ se encuentra tres órdenes de magnitud por encima de la cota de estabilidad.
 - Al contar con un espacio de búsqueda 18 veces mayor, cada par (s, a) se visita con una frecuencia mucho menor durante 1000 episodios, haciendo que las actualizaciones con un α constante y excesivo generen oscilaciones caóticas y divergencia en las aproximaciones de valor en lugar de un aprendizaje asintótico.
3. **Evidencia empírica de ablación (G6) y deficiencia de cobertura ($\epsilon = 0.05$):** El experimento de ablación controlado del Grupo 6 valida de forma empírica la incapacidad del optimizador original para alcanzar la política esperada:
 - Al ejecutar el entrenamiento aplicando **únicamente la recompensa corregida de G2** pero manteniendo el algoritmo original ($\alpha = 0.9$, $\epsilon = 0.05$), el agente no logró acercarse a la cota óptima ($+144.40$): la recompensa acumulada colapsó a un promedio en cola de

−627.73, con un error TD absoluto terminal inestable de 35.80 y una varianza masiva de 39 849.11.

- La política colapsó de manera prematura a una regla subóptima y pasiva: la acción dominante pasó a ser pedir 10 unidades en apenas el 25.5 % de los estados visitados, dejando al resto del espacio sin converger adecuadamente.
- Bajo $\epsilon = 0.05$, el agente actúa de manera puramente *greedy* el 95 % del tiempo. Con un horizonte de 30 pasos por episodio, el Grupo 1 demostró que se requerirían aproximadamente 18 000 episodios (en lugar de los 1 000 asignados) para conservar una densidad de visitas equiparable por estado. Con una tasa estática tan baja, los estados críticos y de demanda alta rara vez se visitan, perpetuando desabastos en la inferencia en producción.

Corregir el MDP y la recompensa repara exclusivamente el modelado del entorno y la métrica de desempeño (el “qué” optimizar), pero deja inalterada la dinámica del algoritmo y la exploración (el “cómo” aprender). Sin un algoritmo numéricamente estable (mediante el decaimiento de α propuesto por el Grupo 3 o el paso normalizado LMS de nuestro Grupo 4) y sin un mecanismo de exploración eficiente en espacios de alta dimensionalidad (Grupo 5), el agente es incapaz de converger y termina atrapado en políticas degeneradas.

2.2. Pregunta 2: Causa raíz

Tomando como problema principal los 23 desabastos semanales, la causa raíz que más contribuye al gap entre simulación y producción es la **cobertura insuficiente del espacio estado–acción**. Esto se debe a que el agente fue evaluado dentro del mismo entorno simplificado donde se entrenó, pero en producción recibió estados y combinaciones de acciones que no había aprendido. Por ende, la recompensa 51.2 de simulación no garantiza que el agente sepa actuar en los estados importantes de producción.

El Grupo 3 encontró que el sistema realmente puede alcanzar 4,371 estados y 26,226 pares estado–acción, no solamente los 200 estados declarados. Sobre ese espacio, el algoritmo original cubrió 1,370.8 pares en promedio, equivalente a solo 5.23 %. Con todas las correcciones del algoritmo la cobertura subió a 2,154.2 pares, pero todavía representa únicamente 8.21 %. Esto demuestra que, aunque el algoritmo mejoró, la mayor parte del espacio continuó sin aprenderse.

El resultado del Grupo 3 también permite descartar que el cambio entre Q-learning y SARSA sea por sí solo la causa principal. El sistema original capturó 30.63 % del valor óptimo y cambiar únicamente a SARSA produjo 33.81 %. En cambio, el desempate aleatorio produjo 68.46 % y aumentar la exploración inicial produjo 73.11 %. Esto se debe a que esas dos correcciones permiten probar más alternativas y evitan que una acción quede seleccionada solamente por el orden de los índices.

El Grupo 5 llegó al mismo resultado usando su entorno discretizado de 200 estados y 1,200 pares. Con $\epsilon = 0.05$ se visitaron 770 pares, equivalente a 64.2 %, y quedaron 430 sin visitar. El problema fue mayor en demanda crítica: quedaron sin visitar 150 de sus 300 pares posibles, es decir, la mitad.

Indicador	Simulación	Producción	Diferencia observada
Recompensa	Media 51.2 y desviación 0.9	No existe una medida equivalente	El valor parece estable, pero no representa directamente ningún indicador del negocio.
Desabastos	El evaluador original no los cuenta; en la simulación del reward corregido se obtuvo 0	23 por semana	La simulación no detectó el principal fallo observado en producción.
Productos vencidos	No se cuentan unidades; solo se aplica una penalización fija	41 por semana	No existe una métrica simulada equivalente para comparar.
Costo de almacenamiento	No se calcula en dinero	\$8,400 frente a una meta de \$3,200	El costo real fue \$5,200 mayor y llegó a 2.625 veces el objetivo.
Satisfacción	No se mide	0.61	La simulación no permite comprobar este resultado antes del despliegue.

Cuadro 7: Diferencia entre las métricas reportadas en simulación y producción.

Además, se encontraron 6 estados donde la política causaba un desabasto directo y 5 de ellos habían probado tres o menos de las seis acciones.

Cuando el Grupo 5 utilizó inicialización optimista más UCB, la cobertura subió a 1,195 de 1,200 pares, equivalente a 99.6 %, y únicamente quedaron 5 pares sin visitar. Por ende, existe evidencia directa de que cambiar la estrategia de exploración casi elimina el problema de cobertura dentro de ese entorno.

Los porcentajes de los Grupos 3 y 5 no se deben comparar directamente. El Grupo 3 conserva la transición sin discretizar y trabaja con 4,371 estados, mientras que el Grupo 5 redondea los estados y trabaja con 200. Sin embargo, los dos experimentos muestran la misma situación: existen muchas acciones que el agente no prueba y la falta de cobertura se concentra en los casos de mayor riesgo.

Esta falta de cobertura se convierte en desabasto debido a la forma en que se inicializa la tabla. Para un estado que nunca fue visitado se tiene:

```

1 Q[state] = [0, 0, 0, 0, 0, 0]
2 action = np.argmax(Q[state]) # devuelve el indice 0: pedir 0

```

Como todas las acciones tienen el mismo valor, `np.argmax` selecciona el primer índice. En el sistema, ese índice significa pedir 0 unidades. Por lo tanto, cuando aparece un estado desconocido con poco inventario y demanda alta o crítica, el agente no realiza un pedido y se puede generar el `stockout`.

El reward original sí tiene problemas y explica el incentivo de almacenar mucho, pero no se selecciona como causa raíz de los desabastos. La evidencia usada para descartarlo como motivo principal es que, en la simulación de la recompensa corregida, tanto la política de pedir siempre 50 como la política corregida obtuvieron cero `stockouts`. Cambiar el reward redujo el inventario promedio de 97.98 a 5.76 unidades y cambió la recompensa de pedir siempre 50 a -842.29 , pero no reprodujo los 23 desabastos de producción.

En conclusión, si se debe escoger un solo motivo, se escoge la **cobertura insuficiente**. El algoritmo original, la exploración baja y el tamaño real del espacio provocan que muchos pares estado-acción no sean aprendidos. Cuando el agente llega a esos estados, la tabla conserva ceros y la política elige pedir 0. Esta es la explicación que conecta de forma más directa los resultados del Grupo 3, los

resultados del Grupo 5 y el gap identificado por el Grupo 7. El código permite demostrar el mecanismo, aunque no permite afirmar que los 23 casos semanales provengan exactamente de 23 estados específicos.

2.3. Pregunta 3: Plan de corrección mínimo viable

Grupos consultados: 1 (MDP corregido), 2 (recompensa corregida), 3 (algoritmo corregido y esquema de decaimiento), 5 (métricas de cobertura), 6 (diagnóstico y proyección de curvas) y 7 (análisis del gap y protocolo de evaluación).

Criterio de “mínimo viable”

El gap de producción tiene tres síntomas cuantificados: sobre-stock (costo de almacenamiento 8,400/semana contra un objetivo de 3,200, es decir $2,6\times$), merma (41 productos vencidos/semana) y desabasto (23 stockouts/semana, satisfacción del cliente 0,61), frente a una recompensa en simulación de 51,2. Las Preguntas 1 y 2 establecieron que la causa no es un solo componente: la recompensa fija el objetivo equivocado (*reward hacking*, Grupo 2) y el optimizador es numéricamente incapaz de alcanzar cualquier objetivo ($\alpha = 0,9$ contra $\alpha_{\text{máx}} = 1,708 \times 10^{-4}$, Grupos 3 y 4).

El plan mínimo viable es el subconjunto más pequeño de correcciones que (a) reorienta el objetivo y (b) hace converger al optimizador hacia él, dejando fuera toda mejora que no sea necesaria para cerrar el gap en dos semanas. Se ordena por *leverage por hora de trabajo*: primero los cambios de bajo costo que desbloquean a los demás.

Las cuatro correcciones del plan

Cronograma de dos semanas

1. **Semana 1.** Días 1–2: implementar y probar la recompensa de G2 (P1a); es un cambio local y sin dependencias. Días 1–5 (en paralelo): corregir `train`, introducir el decaimiento $\alpha_t = \alpha_0/(1 + t/\tau)$ y ε_t decreciente de G3 (P1b). Días 4–5: primer re-entrenamiento con features originales para aislar el efecto de P1 y comparar contra la ablación de G6.
2. **Semana 2.** Días 6–9: integrar `preprocess_state` de 12 características y el paso normalizado (P2); actualizar `n_features` y verificar que `predict/update` no cambian de firma. Días 9–10: activar la inicialización optimista (P3). Días 11–13: re-entrenamiento completo con las cuatro correcciones. Día 14: validación con el protocolo de evaluación de G7 (pruebas de estrés sobre estados de demanda crítica y vencimiento cercano) y curvas de G6.

Qué queda fuera del plan mínimo y por qué

- **El MDP de cinco variables completo de G1.** La expansión completa lleva el espacio de 200 a 3,600 estados ($\times 18$) y de 1,200 a 21,600 pares (s, a) ; G1 estima que se requerirían $\sim 18,000$ episodios para conservar la densidad de visitas, incompatible con dos semanas. Se incorpora *solo* la variable `unidades_en_transito` ($\times 6$, $\rightarrow 1,200$ estados), que es la mínima necesaria para restaurar la propiedad de Markov (entregable 1.1 de G1) y eliminar la ambigüedad de transición que alimenta los stockouts. La `tendencia_demanda` (demanda estocástica, entregable 1.2) se difiere a una fase 2: mejora el realismo pero el gap actual se explica sin ella.
- **Aproximación no lineal.** El Entregable 4.3 mostró $R^2 = 0,967$ sobre estados no vistos con características lineales diseñadas; una red no aporta lo suficiente para justificar su costo.

P	Cambio	Componente que modifica	que	Métrica y mejora esperada
1	Recompensa corregida de tres componentes (Grupo 2)	<code>reward()</code> ; misma firma, ~ 10 líneas		Costo almacenamiento $8,400 \rightarrow \sim 3,200/\text{sem}$ (-62%); vencidos $41 \rightarrow \sim 2/\text{sem}$ (-95%). La política “pedir 50” pasa de óptima a acumular $-842,3$ ($\sigma = 476,2$); la política balanceada alcanza $+144,4$.
1	Corrección de <code>train</code> + decaimiento de α y ε (Grupo 3), o el paso LMS normalizado del Entregable 4.3	<code>train()</code> , hiperparámetros		Error TD terminal $35,8 \rightarrow < 5$; varianza entre episodios $39,849 \rightarrow < 100$ (proyección Grupo 6). Sin este cambio, el Grupo 6 midió que aun con la recompensa de G2 la recompensa de cola queda en $-627,7$ y la política colapsa a “pedir 10” en el 25,5 % de los estados.
2	<code>preprocess_state</code> mejorado: normalización + intercepto + interacciones (Entregable 4.3)	<code>preprocess_state()</code> , <code>n_features</code> $3 \rightarrow 12$		MSE de prueba de $2,76 \times 10^6$ a 678 ($-99,98\%$); R^2 de prueba de $-132,7$ a 0,967; paso admisible de $1,7 \times 10^{-4}$ a 0,337. La política greedy deja de ordenar las acciones al azar en los estados de inventario bajo, donde ocurren los stockouts.
3	Inicialización optimista $w_{a,0} = q_{\text{init}} \approx 4200$ (Grupo 5; habilitada por el intercepto de P2)	Inicialización <code>weights</code> / tabla <code>Q</code>	de	Cobertura de pares (s, a) y reducción de los stockouts atribuibles a estados no visitados (Grupos 5 y 7). Costo de implementación casi nulo y sin riesgo en producción: no explora físicamente, solo cambia el orden de visita en simulación.

Cuadro 8: Plan mínimo viable. Columna **P**: prioridad. Las dos correcciones de prioridad 1 son requisitos: cada una por separado no cierra el gap (Preguntas 1 y 2).

- **UCB o exploración por novedad.** La inicialización optimista entrega la mayor parte del beneficio de cobertura a una fracción del costo de implementación y sin riesgo operativo.

Resultado agregado esperado

Con las cuatro correcciones aplicadas se proyecta (a confirmar contra las curvas finales de G6 y el protocolo de G7): stockouts $23 \rightarrow < 5/\text{semana}$; costo de almacenamiento $8,400 \rightarrow \sim 3,400/\text{semana}$ (dentro de $\sim 1,05\times$ el objetivo); vencidos $41 \rightarrow < 5/\text{semana}$; satisfacción $0,61 \rightarrow > 0,90$. El gap simulación–producción se cierra porque el protocolo de G7 lo detecta con pruebas de estrés sobre estados raros antes del despliegue, no porque desaparezca por sí solo.

2.4. Pregunta 4: Compatibilidad de correcciones

Las correcciones propuestas por los grupos 1, 3 y 4 **no son compatibles de forma directa**. Aunque todas buscan corregir el mismo sistema, cada grupo supone una interfaz y una representación

distintas. Por ello, no basta con copiar los cambios en un solo archivo: se necesita una capa de adaptación que unifique el estado, las acciones y la forma de estimar $Q(s, a)$.

Combinación	Compatibilidad	Adaptación necesaria
Grupo 1 + Grupo 3 tabular	Parcial	Crear un entorno común que devuelva el estado de cinco elementos, convierta el índice de acción a unidades y adapte el retorno adicional de la transición del Grupo 1. La tabla Q acepta la tupla ampliada como llave, pero requerirá mayor cobertura.
Grupo 1 + Grupo 4	No directa	Ampliar <code>preprocess_state</code> para aceptar los cinco componentes e incorporar el pedido en tránsito y la tendencia como características. No se deben eliminar esas variables mediante una proyección al estado antiguo.
Grupo 3 tabular + Grupo 4	No directa	Sustituir <code>Q[state][action]</code> por <code>predict</code> y la actualización tabular por <code>approximator.update</code> ; debe existir una sola representación de Q .
Grupo 3 lineal + Grupo 4	Sí, con adaptación	Reemplazar el preprocesamiento simplificado usado en los experimentos del Grupo 3 por las características del Grupo 4, conservando SARSA, el desempate aleatorio y los calendarios de α y ϵ .
Grupos 1 + 3 + 4	Sí, conceptualmente	Corregir primero la dinámica de los días, construir el entorno común, ampliar las características y entrenar SARSA sobre el aproximador lineal. Después se deben repetir todas las métricas.

Cuadro 9: Compatibilidad y adaptación de las correcciones de los Grupos 1, 3 y 4.

Incompatibilidad en la representación del estado

El Grupo 4 representa cada estado mediante **tres** componentes:

$$s_4 = (\text{inventario}, \text{dias_a_expirar}, \text{nivel_de_demanda}).$$

En cambio, la transición corregida por el Grupo 1 utiliza cinco:

$$s_1 = (\text{inventario}, \text{dias_a_expirar}, \text{nivel_de_demanda}, \text{pedidos_en_transito}, \text{tendencia}).$$

La diferencia no es solamente de longitud. Los pedidos en tránsito y la tendencia contienen información que modifica la dinámica futura; eliminarlos puede hacer que dos situaciones diferentes parezcan el mismo estado para el agente. Además, el entorno del Grupo 3 espera estados de tres elementos, por lo que recibir la tupla de cinco elementos producida por el Grupo 1 causaría errores al desempaquetar el estado o descartaría información relevante.

La solución es definir un entorno adaptador con un contrato único. Su método `reset()` debe devolver siempre el estado de cinco elementos y `step(action_idx)` debe traducir el índice discreto a la acción usada por el Grupo 1. El siguiente fragmento muestra la estructura requerida; los nombres de las funciones de transición y recompensa deben sustituirse por los del módulo integrado.

Listing 3: Adaptador entre el entrenamiento y la dinámica del Grupo 1

```

1 class EntornoFarmaciaCorregido:
2     def __init__(self, max_steps):

```

```

3         self.max_steps = max_steps
4         self.state = None
5         self.steps = 0
6
7     def reset(self):
8         self.steps = 0
9         self.state = estado_inicial_grupo1() # Tupla de 5 elementos
10        return self.state
11
12    def step(self, action_idx):
13        action = ACCIONES[action_idx]
14        state = self.state
15        next_state, faltante = transition(state, action)
16        reward = reward_adaptada(state, action, next_state,
17                                faltante)
18
19        self.steps += 1
20        done = self.steps >= self.max_steps
21        self.state = next_state
22        info = {
23            "accion_aplicada": action,
24            "stockout": faltante,
25            "truncated": done,
26        }
27        return next_state, reward, done, info

```

Este adaptador también evita mezclar un índice de acción, usado por el algoritmo, con la cantidad real solicitada, usada por la función de transición. Por ejemplo, el índice 3 no necesariamente significa pedir tres unidades, sino seleccionar `ACCIONES[3]`.

Incompatibilidad en la representación de Q

El entrenamiento tabular del Grupo 3 consulta una entrada independiente para cada par estado-acción, por ejemplo `Q[state][action]`. El Grupo 4, por el contrario, representa el valor mediante un aproximador lineal:

$$\hat{Q}(s, a; w) = w_a^\top \phi(s),$$

y obtiene o modifica el valor mediante `predict` y `update`. En este caso los pesos se comparten entre muchos estados, de modo que no existe una celda `Q[state][action]` que pueda consultarse o actualizarse directamente. Las dos representaciones no deben mantenerse simultáneamente, porque producirían dos estimaciones distintas de la misma función de valor.

Para integrar el estado del Grupo 1, `preprocess_state` debe aceptar sus cinco componentes. Las unidades en tránsito se normalizan con el máximo pedido y la tendencia, que es categórica, se codifica mediante indicadores *one-hot*. También conviene representar el inventario efectivo que estará disponible cuando llegue el pedido:

Listing 4: Preprocesamiento del estado unificado de cinco elementos

```

1 def preprocess_state_integrado(state):
2     inventory, days, demand, in_transit, trend = state
3
4     # Conserva las características corregidas del Grupo 4.
5     features = preprocess_state((inventory, days, demand))
6     daily_demand = DEMAND_MAP[demand]
7
8     # Nuevas características para el MDP del Grupo 1.
9     transit_n = min(max(in_transit / 50.0, 0.0), 1.0)

```

```

10     effective_inv_n = min(100.0, inventory + in_transit) / 100.0
11     trend_down = 1.0 if trend == "bajando" else 0.0
12     trend_up = 1.0 if trend == "subiendo" else 0.0
13     expected_stockout = max(
14         0.0, daily_demand - inventory - in_transit
15     ) / 40.0
16
17     return list(features) + [
18         transit_n,
19         effective_inv_n,
20         trend_down,
21         trend_up,
22         expected_stockout,
23     ]

```

Con esta ampliación, el vector pasa de 12 a 17 componentes. Por ello deben actualizarse `FEATURE_NAMES`, `N_FEATURES` y el tamaño de la matriz de pesos. La función `estado_a_original` del Grupo 1 puede servir como compatibilidad temporal, pero no como integración final: al proyectar de cinco a tres elementos elimina precisamente las dos variables que recuperan información relevante para la propiedad de Markov.

Listing 5: Inicialización coherente del aproximador

```

1 initial_state = env.reset()
2 n_features = len(preprocess_state_integrado(initial_state))
3 approximator = LinearApproximator(
4     n_features=n_features,
5     n_actions=len(ACCIONES),
6 )

```

Conservación de SARSA durante la integración

La lógica de SARSA del Grupo 3 sí puede conservarse: selección ε -greedy, descuento, calendario de exploración y actualización en cada transición. Lo que debe cambiar es el acceso tabular. Cada lectura de `Q[state][action]` se reemplaza por `predict`, y la asignación a la tabla se reemplaza por `approximator.update`. Para SARSA, el objetivo es

$$y = \begin{cases} r, & \text{si se alcanzó un estado terminal real,} \\ r + \gamma \hat{Q}(s', a'; w), & \text{en otro caso.} \end{cases}$$

En este problema, terminar un episodio por límite de tiempo es un truncamiento: la farmacia no deja de operar. Por eso el código integrado utiliza la segunda rama incluso en el último paso del horizonte, tal como propone el Grupo 3.

Listing 6: Núcleo de SARSA con aproximación lineal

```

1 state = env.reset()
2 features = preprocess_state_integrado(state)
3 action_idx = epsilon_greedy(approximator, features, epsilon)
4
5 while True:
6     next_state, reward, done, info = env.step(action_idx)
7     next_features = preprocess_state_integrado(next_state)
8     next_action_idx = epsilon_greedy(
9         approximator, next_features, epsilon
10    )

```

```

11
12 # El horizonte es un truncamiento, no el fin de la farmacia;
13 # por eso se conserva el bootstrap usado por el Grupo 3.
14 target = reward + gamma * approximator.predict(
15     next_features, next_action_idx
16 )
17
18 alpha_t = alpha_por_visitas(state, action_idx)
19 approximator.update(features, action_idx, target, alpha_t)
20
21 if done:
22     break
23 state, features = next_state, next_features
24 action_idx = next_action_idx

```

El `update` del Grupo 4 normaliza internamente el paso por $\|\phi(s)\|^2$. Por lo tanto, `alpha_t` puede ser el valor decreciente por número de visitas propuesto por el Grupo 3 y el paso efectivo es

$$\alpha_{\text{efectivo}} = \frac{\alpha_t}{\|\phi(s)\|^2}.$$

Esto combina el decaimiento temporal del Grupo 3 con la protección frente a escalas desiguales del Grupo 4.

Antes de integrar también debe corregirse el avance del vencimiento en el MDP del Grupo 1. Su implementación resta un día y después redondea al elemento más cercano de $\{1, 7, 14, 30, 60\}$. Como resultado, los cinco valores regresan al mismo nivel y los días nunca avanzan. Se debe mantener el contador real en la transición,

```

1 new_days = max(1, days_to_expiry - 1)

```

y, si se necesita, discretizarlo solamente al crear las características.

En consecuencia, la integración mínima consiste en: (1) corregir la dinámica de los días y unificar el espacio de estados del Grupo 1; (2) usar su transición mediante `EntornoFarmaciaCorregido`; (3) ampliar `preprocess_state` para conservar los cinco componentes; (4) reemplazar `features_normalizadas` y todos los accesos a la tabla Q por el aproximador del Grupo 4; y (5) conservar del Grupo 3 SARSA, el desempate aleatorio, la exploración y los calendarios de aprendizaje.

Después de estas adaptaciones se deben repetir los experimentos. El $R^2 = 0,967$ del Grupo 4 se calculó sobre el MDP determinista original, mientras que el 98,4 por ciento reportado por el Grupo 3 utilizó un preprocesamiento lineal simplificado y no el estado corregido de cinco elementos. Esos resultados son evidencia de que la combinación es prometedora, pero todavía no validan el sistema integrado.

Finalmente, la compatibilidad técnica no corrige la recompensa. Si se conserva la función original, el agente integrado puede aprender de manera más estable y precisa la política de pedir el máximo. Por ello, la recompensa debe corregirse antes del reentrenamiento definitivo.

3. Reflexión grupal

Durante la fase previa de análisis individual, nuestro equipo llegó a la sesión con una hipótesis clara: el mal desempeño del agente se debía principalmente a la deficiente ingeniería de características y a la escala sin normalizar en el preprocesamiento. Al constatar que el inventario concentraba el 75 % de la norma del gradiente frente a apenas el 0.09 % de la demanda, y al comprobar analíticamente que la tasa $\alpha = 0.9$ violaba la condición de estabilidad LMS ($\alpha_{\text{máx}} \approx 1.708 \times 10^{-4}$), asumíamos que corrigiendo la representación lineal e incorporando las interacciones no lineales resolveríamos el problema. Tras verificar que nuestro modelo propuesto reducía el MSE de prueba en un 99.98 %, considerábamos que el cuello de botella estaba resuelto.

El contraste de resultados en la sesión presencial cambió por completo este diagnóstico. Al revisar la función de recompensa con el Grupo 2, descubrimos que el entorno sufría de un severo *reward hacking* que premiaba pedir siempre 50 unidades; por tanto, aunque nuestro aproximador fuera perfecto con un MSE nulo sobre los 4367 estados alcanzables, el agente seguiría aprendiendo una política destructiva. Asimismo, el Grupo 1 nos demostró que el estado original carecía de la propiedad de Markov y que, al corregirlo con el inventario en tránsito y la tendencia de demanda, el espacio de estados se multiplicaba por 18 (3600 estados), obligando a que cualquier esquema de aproximación deba gestionar una dimensionalidad mucho mayor. Finalmente, los experimentos de ablación del Grupo 6 confirmaron que aun aplicando la recompensa corregida, el agente colapsaba a recompensas negativas (-627.73) si no se estabilizaban el algoritmo y la aproximación de forma conjunta.

La lección fundamental de esta auditoría es que en Aprendizaje por Refuerzo no es posible resolver un problema optimizando componentes de forma aislada. La aproximación de funciones no opera en el vacío, sino como el puente que traduce el estado del entorno (MDP) y transmite el gradiente de la señal de desempeño (recompensa). De nada sirve perfeccionar la representación de características si la recompensa premia una conducta errónea o si el entorno oculta variables críticas para la toma de decisiones.

A. Anexo: uso de IA generativa

Se usó IA generativa como apoyo de herramienta, no para el análisis del componente. A continuación los prompts empleados y la respuesta que se obtuvo de cada uno.

Prompt 1: compilación del documento

No tengo pdflatex instalado en el sistema y estoy en NixOS. Necesito compilar informe_grupo4.tex a PDF sin instalar nada de forma permanente. El documento usa babel en español, booktabs, longtable, siunitx, enumitem y listings con lsteinputlisting. Dame el comando exacto con nix-shell -p, indicando qué esquema de TeX Live necesito para que no falte ningún paquete, y explícame por qué hay que correr pdflatex más de una vez.

Respuesta obtenida:

```
nix-shell -p texlive.combined.scheme-full --run \
'pdflatex -interaction=nonstopmode -halt-on-error informe_grupo4.tex && \
pdflatex -interaction=nonstopmode -halt-on-error informe_grupo4.tex'
```

Prompt 2: escritura de las fórmulas

Convierte estas expresiones a $\text{\LaTeX}$ con amsmath, en modo display y tipográficamente correctas: (1) la regla de actualización semi-gradiente para un aproximador lineal $Q(s, a; w) = w_a^\top \phi(s)$, dejando explícito que el gradiente respecto a w_a es $\phi(s)$; (2) la condición de estabilidad de LMS $0 < \alpha < 2/\|\phi(s)\|^2$; (3) el error cuadrático medio promediado sobre estados y acciones. Requisitos: usa $\lVert$ / $\rVert$ y no $||$; nombres de variables del código en $\text{\texttt{}}\text{\texttt{}}$ dentro de modo matemático; y si una línea se pasa del margen, parte la ecuación con $\text{\texttt{gather*}}$ en vez de dejar $\text{\texttt{overfull hbox}}$.

Respuesta obtenida:

Regla de actualización semi-gradiente:

$$w_a \leftarrow w_a + \alpha [y - w_a^\top \phi(s)] \nabla_{w_a} Q(s, a; w), \quad \nabla_{w_a} Q(s, a; w) = \phi(s).$$

Condición de estabilidad de LMS:

$$0 < \alpha < \frac{2}{\|\phi(s)\|^2}.$$

Error cuadrático medio sobre estados y acciones:

$$\text{MSE} = \frac{1}{|S| \cdot |A|} \sum_s \sum_a (Q^*(s, a) - w_a^\top \phi(s))^2.$$